

COE619 PROJECT PROPOSAL

Developing Fully Fledged Quantum Simulator in Java

Omar Alshammari 200819560

KFUPM

EXECUTIVE SUMMARY

Advances in quantum algorithms are often bottlenecked by limited access to physical qubits; high-fidelity, noise-aware classical simulators therefore remain essential for prototyping, benchmarking, and education. This project proposes a fully-fledged, open-source quantum-circuit simulator written in Java and compiled to a highly optimized native binary via GraalVM. By targeting the rich JVM ecosystem—yet escaping its performance overhead at runtime—the simulator aims to combine developer productivity, cross-platform portability, and near-C++ execution speed.

Technically, the simulator will feature a modular architecture with interchangeable back-ends:

- State-vector engine for ideal circuits up to ~30 qubits on commodity hardware.
- Noise engine that injects realistic error channels (depolarizing, amplitude-damping, phase-flip, T_1/T_2 decoherence) at gate-, qubit-, or cycle-granularity, enabling fidelity studies and algorithm robustness tests.
- Stabilizer and tensor-network paths for Clifford and shallow-depth, large-qubit workloads.

Built-in support for parallelism, SIMD intrinsics, and memory-efficient data layouts will be benchmarked against leading simulators such as Qiskit Aer and QuEST.

Key deliverables include:

- GraalVM-native binaries and Gradle artifacts.
- Well-documented Java API and native command-line interface.
- Comprehensive documentation, tutorials, and unit tests.
- A benchmarking report detailing accuracy, scalability, and noise-model fidelity.

The resulting toolchain will empower KFUPM researchers, educators, and the broader Java community to explore—and stress-test—quantum algorithms.

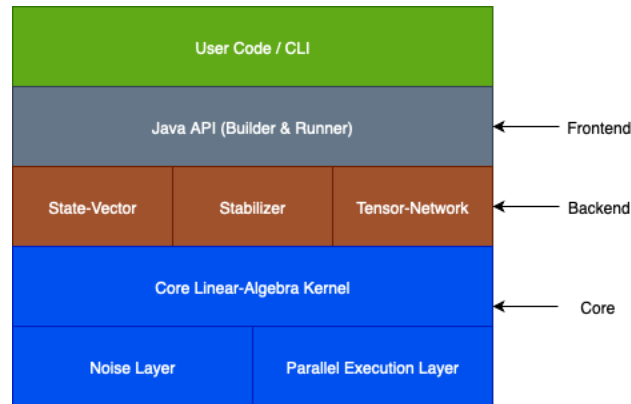
OBJECTIVES

Objective	Details
O-1 Correctness & Scalability	Implement a state-vector back-end that executes universal quantum circuits of ≤ 30 qubits and $\leq 10\,000$ gates
O-2 Noise Simulation	Integrate configurable depolarizing, amplitude-damping, phase-flip, and T_1/T_2 decoherence channels
O-3 Alternative Engines	Provide (a) a stabilizer simulator for Clifford circuits up to 1,000 qubits and (b) an experimental tensor-network path that can handle 50 qubits at depth ≤ 40
O-4 Parallelism & SIMD	Integrate SIMD intrinsics (JDK Vector API)

O-5 Developer Interface	Publish a well-documented Java API and a command-line interface packaged in the native binary.
O-6 Algorithm Case-Study Suite	Implement and document a set of representative quantum algorithms/circuits with clear expected outcomes: - Bell state & GHZ - Deutsch–Jozsa and Bernstein–Vazirani - Quantum Fourier Transform (QFT) & Phase Estimation (small-n)

TECHNICAL APPROACH & METHODOLOGY

SYSTEM ARCHITECTURE



- Front-end – A fluent Java builder (CircuitBuilder) plus a concise CLI that reads OpenQASM 3 or JSON, builds a circuit object, and invokes the selected back-end.
- Back-ends – Pluggable engines share a common SimulatorEngine interface and register via Java’s ServiceLoader.
- Core Kernel – Centralised BLAS-like routines (Kronecker products, matrix-vector multiplies) written in plain Java but leveraging the JDK Vector API.
- Noise Layer – Composable ErrorChannel descriptors injected between gate applications.
- Parallel Layer – Fork-Join tasks or StructuredTaskScope (JDK 21).

IMPLEMENTATION PLAN

WORK BREAKDOWN STRUCTURE

Phase	Term/Week	Core Deliverable
P1 Repo setup, Gradle skeleton, CI, GraalVM toolchain	T3 – W1	Git repo, Gradle skeleton, GraalVM / CI pipeline ready

P2 Core math kernel + circuit parser (OpenQASM3/JSON)	T3 – W5	End-to-end parse→simulate path (small circuits), SIMD spikes
P3 State-vector MVP + Algorithm mini-suite v1	T3 – W8	Progress Report 1
P4 Noise Layer	T3 – W16	Progress Report 2
P5 Stabilizer engine + tensor-network prototype	T4 – W4	Alt-engine demo, Clifford tests
P6 Java API & CLI	T4 – W8	Progress Report 3
P7 Documentation & Tutorials	T4 – W13	Final Report
P8 Final Integration & Review	T4 – W14	v 1.0 release tag, demo, and project presentation

KEY MILESTONES

Milestone	Description
M1 – Core Ready (T3–W5)	Parser + kernel E2E on sample circuits; initial vectorization measured.
M2 – State-Vector Validated (T3–W8)	Algorithm suite v1
M3 – Noise Engine Complete (T3–W16)	Depolarizing, amplitude-damping, phase-flip, T_1/T_2
M4 – Alt-Engine Showcase (T4–W4)	Stabilizer engine passing random Clifford tests; tensor-network demo on $n \approx 30$ –50 shallow depth
M5 – Native & Parallel Gains (T4–W8)	SIMD and parallelization
M6 – Release Candidate (T4–W13)	Final benchmarks + validation report

CONCLUSION

This project charts a clearpath to deliver a high-performance, noise-aware quantum-circuit simulator written in pure Java and compiled to a GraalVM native binary. By focusing on a rigorously validated state-vector core, extensible noise models, alternative stabilizer and tensor-network engines, and a developer-friendly Java API and CLI.

A concise eight-phase work plan with six concrete milestones demonstrates both technical feasibility and time-bounded accountability, ensuring that, by the end of 4th Term, KFUPM researchers and students will have a portable, open-source simulator capable of prototyping and benchmarking quantum algorithms without specialised hardware.